LevelUp AI — Human-Quality Test Case Writing Standard

Version: 1.0
Date: July 9, 2026
Status: Canonical

Purpose

This document defines **what a senior QA engineer writes** when creating manual test cases.

This is **not prompt engineering**.

This is the **authoritative contract** for test case quality across all LevelUp AI systems.

Once this standard is fixed, every LLM prompt, template, validator, and future model must follow it.

Scope

This standard applies to:

- ✅ Test Case Lab (generation)
- ✅ Script Generation (consumption)
- ✅ Manual Test Export
- ✅ AI Review / Validation
- ✅ Healing Validation
- ✅ Future Jira / ALM Export

One definition. Everywhere.

The 15 Principles

Principle 1 — One Test Case = One Objective

Bad

```
Verify invalid username and invalid password.
```

Good

```
Verify login fails with an invalid password.
```

Rule: Only ONE variable changes per test case.

Principle 2 — Every Step Has One Action

Bad

```
Enter username and password.
```

Good

```
1. Open Login page.
2. Enter registered email.
3. Enter valid password.
4. Click Login.
```

Rule: Never combine actions. Each step is atomic.

Principle 3 — User Actions Only

Bad

```
Ensure login button is clickable.
```

Good

```
Click Login.
```

Bad

```
Observe the page.
```

Good

```
Verify Home page is displayed.
```

Rule: If a user cannot physically perform it, it shouldn’t be an action step.

Principle 4 — Verification ≠ Action

Separate them.

Bad

```
Click Login and verify Home page.
```

Good

```
4. Click Login.
5. Verify Home page is displayed.
```

Rule: Actions and assertions are distinct steps.

Principle 5 — Business Language

Never expose automation vocabulary.

Bad

```
Fill username field.
```

Good

```
Enter registered email address.
```

Bad

```
Trigger submit.
```

Good

```
Click Login.
```

Rule: Use the language a business analyst or product owner would use.

Principle 6 — Observable Expected Results

This is the biggest quality improvement.

Never write:

```
Login successful.
```

Always write:

```
Home page is displayed.
Logged-in username is visible.
Logout button is available.
Login form is no longer displayed.
```

Rule: Everything must be **observable** by a human tester. No abstract success criteria.

Principle 7 — Test Data Never Inside Steps

Never:

```
Enter standard_user.
Enter secret_sauce.
```

Always:

```
Enter registered username.
Enter valid password.
```

Then provide metadata:

```
Test Data

Role: Registered User
Dataset: valid_users
Record: standard_user
```

Rule: Steps describe the role of the data. The Dataset Resolver maps role → actual data downstream.

Principle 8 — Human Flow

Every test case follows the same rhythm:

```
Setup
  ↓
Action
  ↓
Verification
```

Never:

```
Setup
  ↓
Verification
  ↓
Action
  ↓
Verification
```

Rule: Execution order must be natural and sequential.

Principle 9 — Preconditions Are NOT Steps

Wrong:

```
1. User is registered.
2. Login page is open.
3. Enter username.
```

Correct:

```
Preconditions
• Registered user exists.
• Application is available.

Steps
1. Open Login page.
2. Enter registered email.
```

Rule: Preconditions describe the starting state. Steps describe user actions.

Principle 10 — Test Data Should Explain Intent

Not:

```
Dataset: valid_users
```

Instead:

```
Role: Registered User
Purpose: Valid credentials for successful authentication
```

Rule: Data metadata should clarify why this data is used, not just what it is.

Principle 11 — Scenario Title Formula

Every title follows one pattern:

```
Verify <expected behavior> when <condition>.
```

Examples:

```
Verify successful login with valid credentials.
Verify login fails with an invalid password.
Verify validation message when password is empty.
Verify account lock after five failed attempts.
```

Rule: No creativity. Consistent, predictable titles.

Principle 12 — Priority Should Be Risk-Based

Not:

```
Positive = High
Negative = Medium
```

Instead:

Business impact determines priority.

```
Authentication       → High
Payment              → Critical
Forgot Password      → Medium
UI Styling           → Low
```

Rule: Priority reflects business risk, not test type.

Principle 13 — Every Test Case Ends the Same Way

A QA engineer should immediately understand **Pass ↔ Fail** without interpretation.

Never:

```
System behaves correctly.
```

Always:

```
User remains on Login page.
Authentication error message is displayed.
Session is not created.
```

Rule: Expected results must be specific, observable, and unambiguous.

Principle 14 — No Assumptions

If the requirement says:

```
Login
```

Don’t generate:

```
Locked User
Remember Me
OTP
Session Timeout
Captcha
```

Unless justified by:
- Requirement text
- Acceptance criteria
- App knowledge (repository intelligence)
- Scenario obligation (KB mandate)

Rule: Only test what is justified. No creative expansion.

Principle 15 — Senior QA Review Test

Every generated test case should pass these 10 checks:

Question	Pass?
One objective?	✅
One variable changed?	✅
Human wording?	✅
No automation wording?	✅
Observable results?	✅
Correct test data role?	✅
No assumptions?	✅
Steps in execution order?	✅
One action per step?	✅
Can Script Gen consume directly?	✅

Rule: If any check fails, the test case does not meet the standard.

Implementation Guidance

For LLM Prompts

When instructing an LLM to generate test cases:
1. Reference this standard explicitly.
2. Provide examples that follow these principles.
3. Include the 10-question review checklist in the system prompt.

For Validators

Automated validators should enforce:
- One action per step (no “and” or “then” within a single step)

- No hardcoded test data in steps
- Observable expected results (avoid “successful”, “correct”, “works”)
- Preconditions vs. steps separation

For Exporters

When exporting to Jira, CSV, or other formats:
- Preserve the structure (Preconditions / Steps / Expected Results)
- Keep data roles and dataset metadata separate from steps
- Maintain the title formula

Examples

✅ Good — Valid Login

Title: Verify successful login with valid credentials.

Preconditions:
- Registered user exists.
- Application is available.

Steps:
1. Open Login page.
2. Enter registered email.
3. Enter valid password.
4. Click Login.

Expected Results:
- Home page is displayed.
- Logged-in username is visible in the header.
- Logout button is available.
- Login form is no longer displayed.

Test Data:
- Role: Registered User
- Purpose: Valid credentials for successful authentication
- Dataset: valid_users

✅ Good — Invalid Password

Title: Verify login fails with an invalid password.

Preconditions:
- Registered user exists.
- Application is available.

Steps:
1. Open Login page.
2. Enter registered email.
3. Enter incorrect password.
4. Click Login.

Expected Results:
- User remains on Login page.
- Authentication error message is displayed.
- Error message does not reveal whether the username exists.
- Session is not created.

Test Data:
- Role: Registered User
- Purpose: Valid username paired with an incorrect password
- Dataset: valid_users
- Variation: Password replaced with an invalid value

❌ Bad — Multiple Objectives

Title: Verify login and password reset.

Steps:
1. Enter invalid credentials.
2. Click Login.
3. Click Forgot Password.
4. Enter email.
5. Verify reset email sent.

Why it’s bad:
- Two objectives (login failure + password reset).
- Violates Principle 1 (One Test Case = One Objective).

❌ Bad — Combined Actions

Steps:
1. Open Login page and enter username and password.
2. Click Login and verify Home page.

Why it’s bad:
- Multiple actions per step (violates Principle 2).
- Action and verification combined (violates Principle 4).

❌ Bad — Hardcoded Data

Steps:
1. Enter standard_user.
2. Enter secret_sauce.
3. Click Login.

Why it’s bad:
- Test data is embedded in steps (violates Principle 7).
- Steps are not reusable across different applications or datasets.

Relationship to Other Standards

ScenarioSemantics (Sprint 2A)

This standard consumes `ScenarioSemantics` from the KB to produce human-quality test cases.

Mapping:
- `variableUnderTest` → informs which field changes (Principle 1)
- `preconditions` → maps to “Preconditions” section (Principle 9)
- `variation` → informs how the step is worded (single-variable principle)
- `expectedBehavior` → maps to “Expected Results” section (Principle 6)
- `requiredDataRole` → maps to “Test Data” section (Principle 7)

Script Generation (Sprint 2D)

Script Generation must:
- Parse steps written to this standard.
- Map data roles to actual datasets (via Dataset Resolver).
- Translate human actions (“Click Login”) to Playwright code (`loginPage.clickLogin()`).

Dataset Resolver (Sprint 2C)

The Dataset Resolver maps:
- `requiredDataRole` (e.g., `registered_user`) → actual dataset (e.g., `valid_users.csv`)
- `variation` (e.g., “incorrect password”) → data mutation strategy

Enforcement

This standard is **mandatory** for:
- All Test Case Lab output.
- All manual test case exports.
- All human-facing QA artifacts.

If a generated test case does not pass the 15 principles + 10-question checklist, it is **not production-ready**.

Version History

Version	Date	Changes
1.0	2026-07-09	Initial standard (Sprint 2B)

Maintainer

This is a **living document**. If a principle is ambiguous or a new pattern emerges, update this standard first, then adjust implementation.

Owned by: LevelUp AI Product / Engineering
Contact: Prasanth (LevelUp), Abacus AI Agent

End of Standard